Solution: 0/1 Knapsack

Introduction

Given the weights and profits of 'N' items, we are asked to put these items in a knapsack that has a capacity 'C'. The goal is to get the maximum profit from the items in the knapsack. Each item can only be selected once, as we don't have multiple quantities of any item.

Let's take Merry's example, who wants to carry some fruits in the knapsack to get maximum profit. Here are the weights and profits of the fruits:

Items: { Apple, Orange, Banana, Melon }
Weights: { 2, 3, 1, 4 }
Profits: { 4, 5, 3, 7 }
Knapsack capacity: 5

Let's try to put different combinations of fruits in the knapsack, such that their total weight is not more than 5:

Apple + Orange (total weight 5) => 9 profit
Apple + Banana (total weight 3) => 7 profit
Orange + Banana (total weight 4) => 8 profit
Banana + Melon (total weight 5) => 10 profit

This shows that **Banana + Melon** is the best combination, as it gives us the maximum profit and the total weight does not exceed the capacity.

Problem Statement

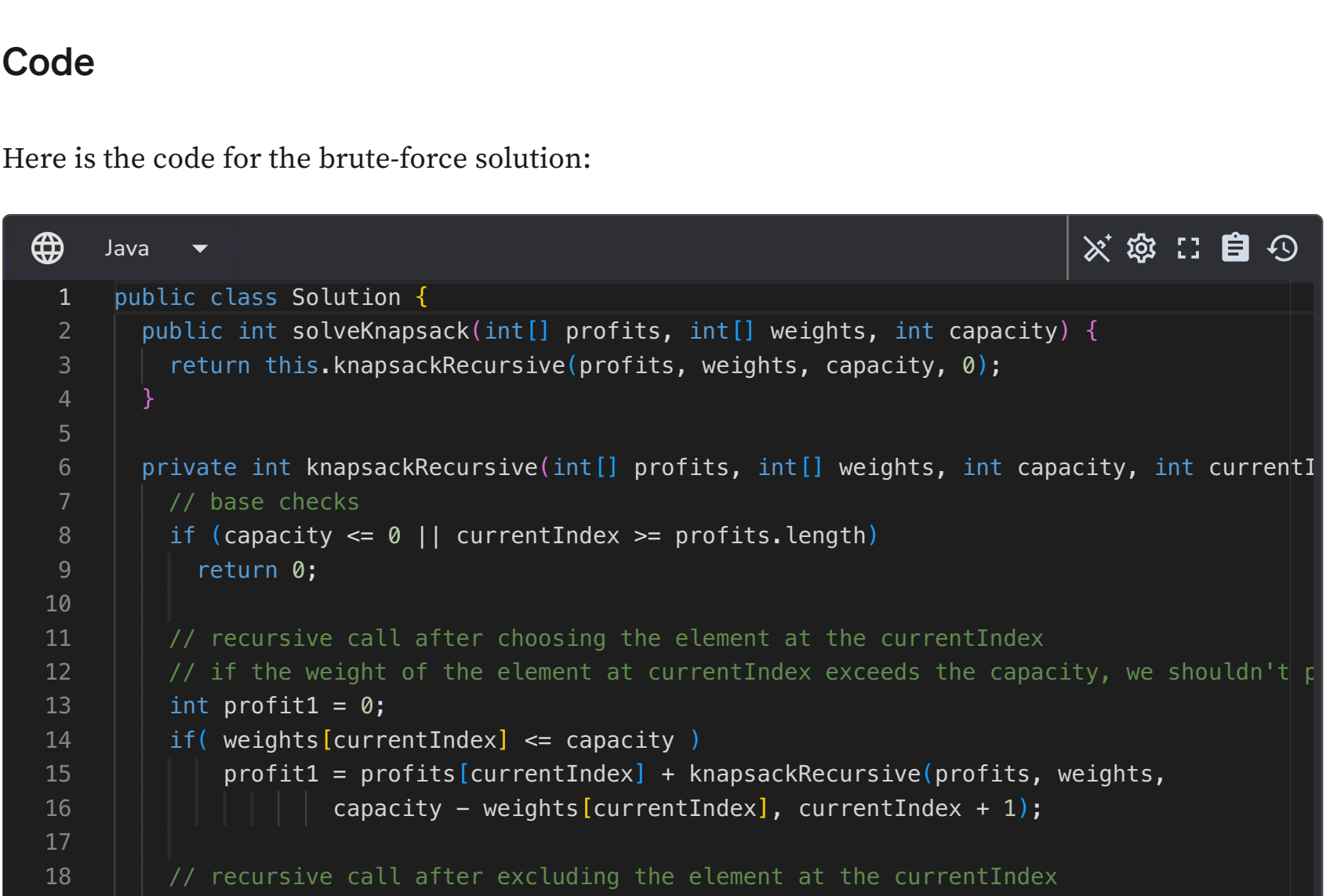
Given two integer arrays to represent weights and profits of 'N' items, we need to find a subset of these items which will give us maximum profit such that their cumulative weight is not more than a given number 'C'. Write a function that returns the maximum profit. Each item can only be selected once, which means either we put an item in the knapsack or skip it.

Basic Solution

A basic brute-force solution could be to try all combinations of the given items (as we did above), allowing us to choose the one with maximum profit and a weight that doesn't exceed 'C.' Take the example of four items (A, B, C, and D), as shown in the diagram below. To try all the combinations, our algorithm will look like:

```
for each item 'i'
  create a new set which INCLUDES item 'i' if the total weight does not exceed the
capacity, and
     recursively process the remaining capacity and items
  create a new set WITHOUT item 'i', and recursively process the remaining items
return the set from the above two sets with higher profit
```

Here is a visual representation of our algorithm:


items A B C D
profit 1 6 10 16
weight 1 2 3 5
Capacity: 7
values: [A, B, C, D] selected: [] profit: 0, weight:0
select A
skip A
values: [B, C, D] selected: [A] profit: 1, weight:1
values: [B, C, D] selected: [] profit: 0, weight:0
select B
skip B
select B
skip B
values: [C, D] selected: [A, B] profit: 7, weight:3
values: [C, D] selected: [A] profit: 1, weight:1
values: [C, D] selected: [B] profit: 6, weight:2
values: [C, D] selected: [] profit: 0, weight:0
select C
skip C
select C
skip C
select C
skip C
select C
skip C
values: [D] selected: [A, B, C] profit: 17 weight: 6
values: [D] selected: [A, B] profit: 7 weight:3
values: [D] selected: [A, C] profit: 11 weight:4
values: [D] selected: [A] profit: 1 weight:1
values: [D] selected: [B, C] profit: 16 weight:5
values: [D] selected: [B] profit: 6 weight:2
values: [D] selected: [C] profit: 10 weight:3
values: [D] select: [] profit: 0 weight:0
select D
skip D
values: [] select: [A, B, C, D] profit: 33 weight: 11
values: [] select: [A, B, C] profit: 17 weight: 6
values: [] select: [A, B, D] profit: 23 weight: 8
values: [] select: [A, B] profit: 7 weight: 3
values: [] select: [A, C, D] profit: 27 weight: 9
values: [] select: [A, C] profit: 11 weight: 4
values: [] select: [A, D] profit: 16 weight: 6
values: [] select: [A] profit: 1 weight:1
values: [] select: [B, C, D] profit: 32 weight:10
values: [] select: [B, C] profit: 16 weight: 5
values: [] select: [B, D] profit: 22 weight: 7
values: [] select: [B] profit: 6 weight: 2
values: [] select: [C, D] profit: 26 weight: 8
values: [] select: [C] profit: 10 weight: 3
values: [] select: [D] profit: 16 weight: 5
values: [] select: [] profit: 0 weight:0

Recursion tree for 0/1 Knapsack Problem

All green boxes have a total weight that is less than or equal to the capacity (7), and all the red ones have a weight that is more than 7. The best solution we have is with items [B, D] having a total profit of 22 and a total weight of 7.

Code

Here is the code for the brute-force solution:

Java

```java
public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    return this.knapsackRecursive(profits, weights, capacity, 0);
  }

  private int knapsackRecursive(int[] profits, int[] weights, int capacity, int currentI
    // base checks
    if (capacity <= 0 || currentIndex >= profits.length)
      return 0;

    // recursive call after choosing the element at the currentIndex
    // if the weight of the element at currentIndex exceeds the capacity, we shouldn't p
    int profit1 = 0;
    if( weights[currentIndex] <= capacity )
        profit1 = profits[currentIndex] + knapsackRecursive(profits, weights,
              capacity - weights[currentIndex], currentIndex + 1);

    // recursive call after excluding the element at the currentIndex
    int profit2 = knapsackRecursive(profits, weights, capacity, currentIndex + 1);

    return Math.max(profit1, profit2);
```

Test Case | Run Code Result | Stdout

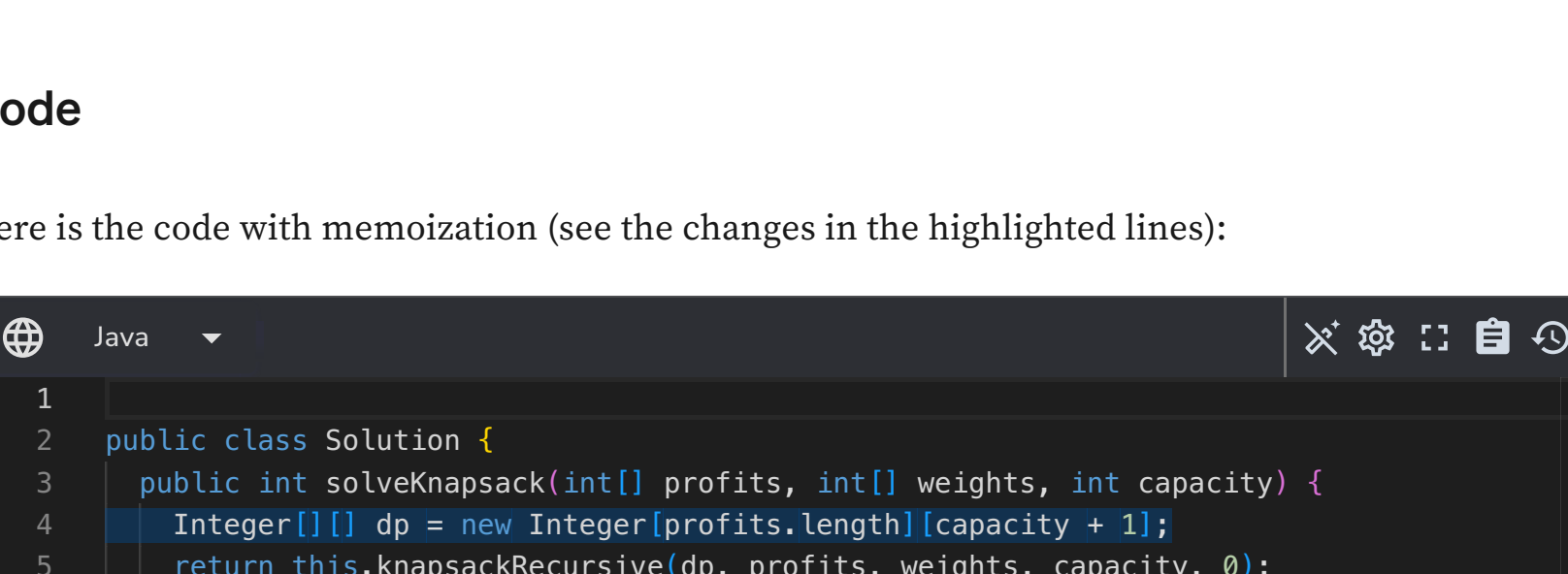
```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

The above algorithm's time complexity is exponential $O(2^n)$, where 'n' represents the total number of items. This can also be confirmed from the above recursion tree. As we can see that we will have a total of '31' recursive calls -- calculated through $(2^n) + (2^n) - 1$, which is asymptotically equivalent to $O(2^n)$.

The space complexity is $O(n)$. This space will be used to store the recursion stack. Since our recursive algorithm works in a depth-first fashion, which means, we can't have more than 'n' recursive calls on the call stack at any time.

Let's visually draw the recursive calls to see if there are any overlapping sub-problems. As we can see, in each recursive call, profits and weights arrays remain constant, and only capacity and currentIndex change. For simplicity, let's denote capacity with 'c' and currentIndex with 'i':



We can clearly see that '**c:4, i=3**' has been called twice; hence we have an overlapping sub-problems pattern. As we discussed above, overlapping sub-problems can be solved through Memoization.

Top-down Dynamic Programming with Memoization

We can use memoization to overcome the overlapping sub-problems. To reiterate, memoization is when we store the results of all the previously solved sub-problems and return the results from memory if we encounter a problem that has already been solved.

Since we have two changing values (`capacity` and `currentIndex`) in our recursive function `knapsackRecursive()` , we can use a two-dimensional array to store the results of all the solved sub-problems. As mentioned above, we need to store results for every sub-array (i.e., for every possible index 'i') and for every possible capacity 'c'.

Code

Here is the code with memoization (see the changes in the highlighted lines):

Java

```java

public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    Integer[][] dp = new Integer[profits.length][capacity + 1];
    return this.knapsackRecursive(dp, profits, weights, capacity, 0);
  }

  private int knapsackRecursive(Integer[][] dp, int[] profits, int[] weights, int capaci
      int currentIndex) {

    // base checks
    if (capacity <= 0 || currentIndex >= profits.length)
      return 0;

    // if we have already solved a similar problem, return the result from memory
    if(dp[currentIndex][capacity] != null)
      return dp[currentIndex][capacity];

    // recursive call after choosing the element at the currentIndex
    // if the weight of the element at currentIndex exceeds the capacity, we s
    int profit1 = 0;
```

Test Case | Run Code Result | Stdout

```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

What is the time and space complexity of the above solution? Since our memoization array `dp[profits.length][capacity+1]` stores the results for all the subproblems, we can conclude that we will not have more than $N * C$ subproblems (where 'N' is the number of items and 'C' is the knapsack capacity). This means that our time complexity will be $O(N * C)$.

The above algorithm will be using $O(N * C)$ space for the memoization array. Other than that, we will use $O(N)$ space for the recursion call-stack. So the total space complexity will be $O(N * C + N)$, which is asymptotically equivalent to $O(N * C)$.

Bottom-up Dynamic Programming

Let's try to populate our `dp[][]` array from the above solution, working in a bottom-up fashion. Essentially, we want to find the maximum profit for every sub-array and for every possible capacity.

This means, `dp[i][c]` will represent the maximum knapsack profit for capacity 'c' calculated from the first 'i' items.

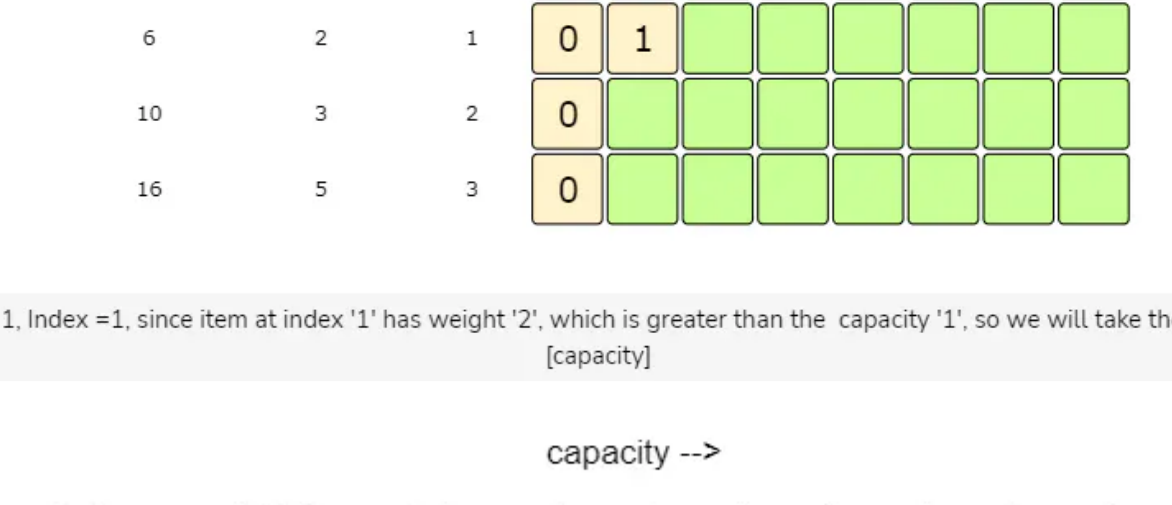
So, for each item at index 'i' (0 <= i < items.length) and capacity 'c' (0 <= c <= capacity), we have two options:

1. Exclude the item at index 'i'. In this case, we will take whatever profit we get from the sub-array excluding this item => `dp[i-1][c]`
2. Include the item at index 'i' if its weight is not more than the capacity. In this case, we include its profit plus whatever profit we get from the remaining capacity and from remaining items => `profits[i] + dp[i-1][c-weights[i]]`

Finally, our optimal solution will be maximum of the above two values:

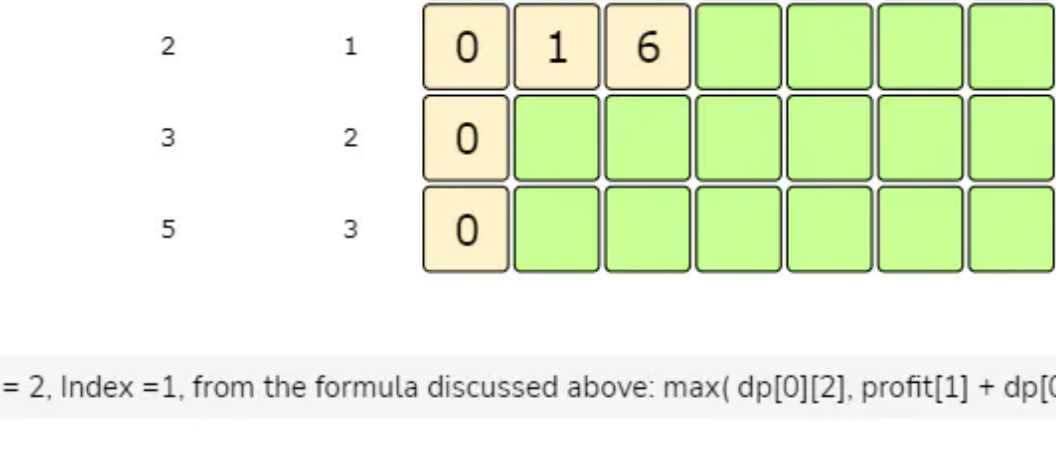
```
dp[i][c] = max (dp[i-1][c], profits[i] + dp[i-1][c-weights[i]])
```

Let's visually draw this and start with our base case of zero capacity:


capacity -->

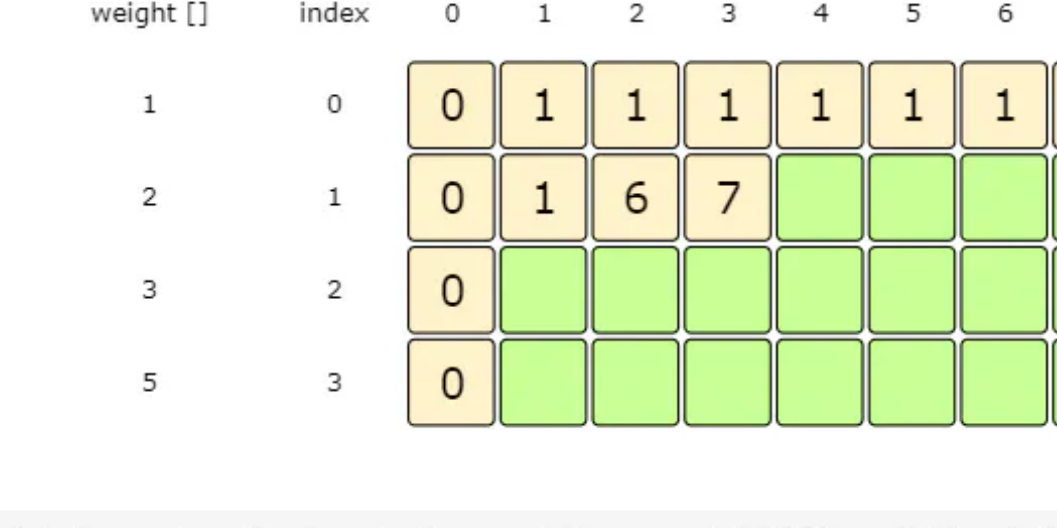
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0							
6	2	1	0							
10	3	2	0							
16	5	3	0							

With '0' capacity, maximum profit we can have for every subarray is '0'


capacity -->

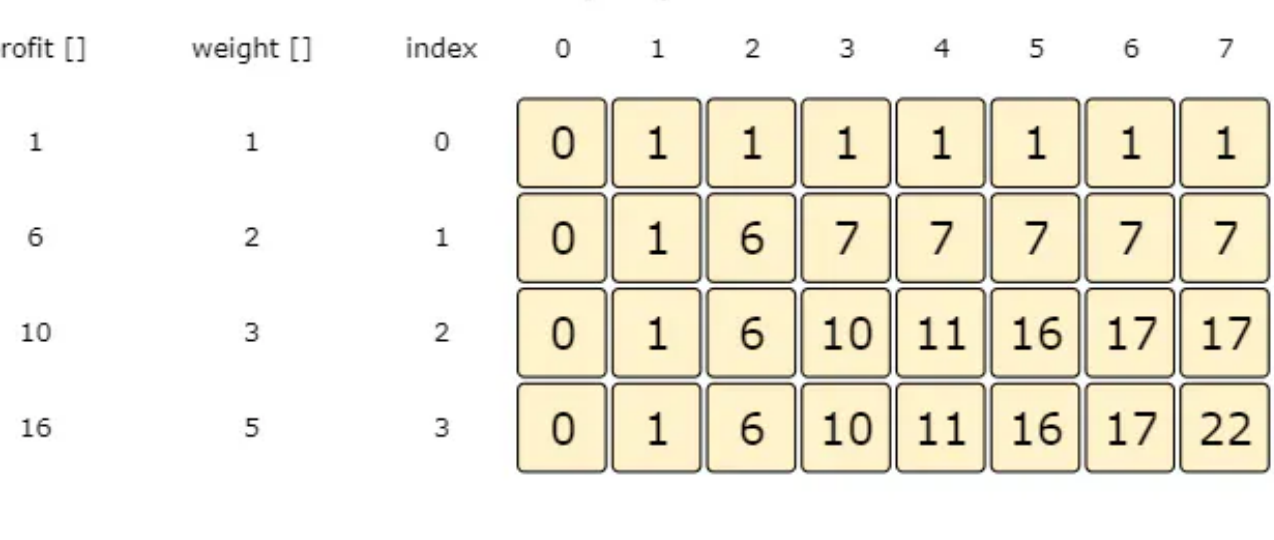
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0							
10	3	2	0							
16	5	3	0							

Capacity = 1-7, Index = 0, i.e., if we consider the sub-array till index '0', this means we have only one item to put in the knapsack, we will take it if it is not more than the capacity


capacity -->

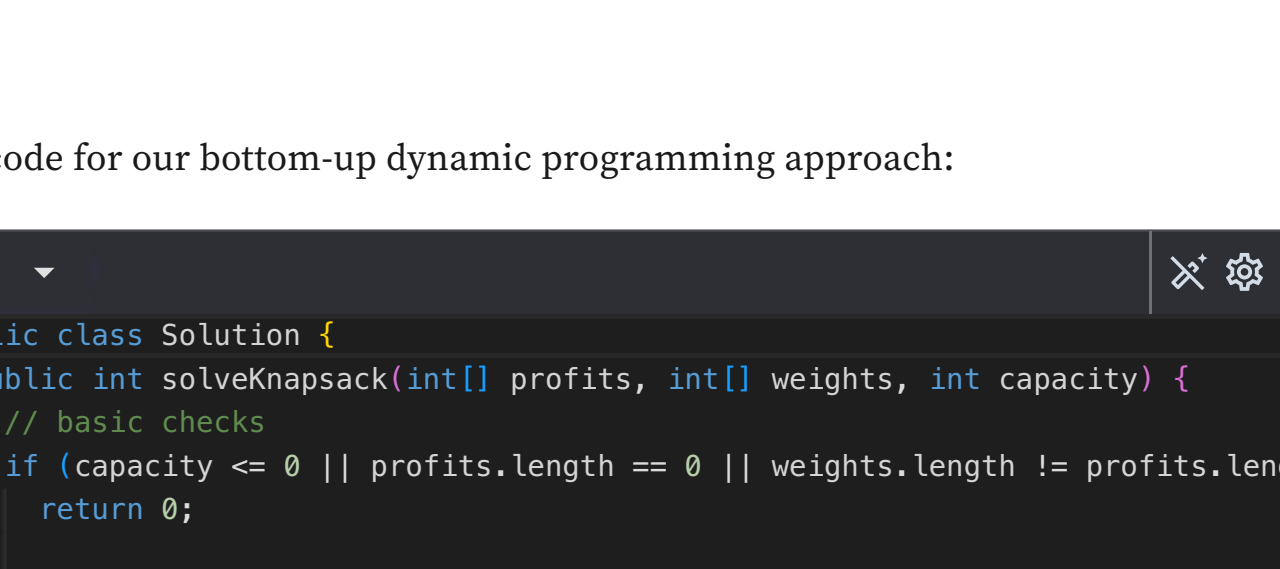
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1						
10	3	2	0							
16	5	3	0							

Capacity = 1, Index =1, since item at index '1' has weight '2', which is greater than the capacity '1', so we will take the dp[index-1][capacity]


capacity -->

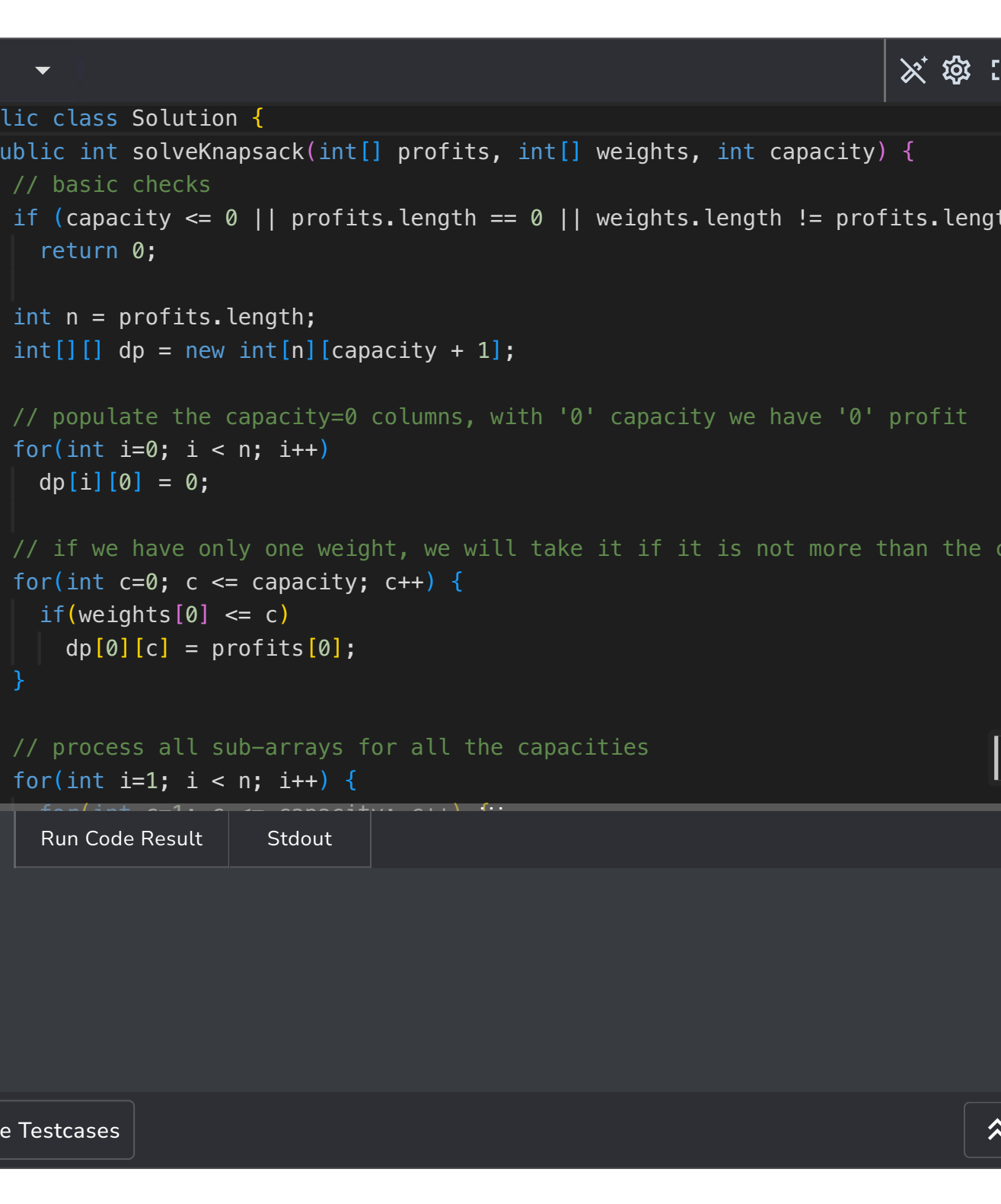
profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1	6					
10	3	2	0							
16	5	3	0							

Capacity = 2, Index =1, from the formula discussed above: max(dp[0][2], profit[1] + dp[0][0])


capacity -->

profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1	6	7				
10	3	2	0							
16	5	3	0							

Capacity = 3, Index =1, from the formula discussed above: max(dp[0][3], profit[1] + dp[0][1])


capacity -->

profit []	weight []	index	0	1	2	3	4	5	6	7
1	1	0	0	1	1	1	1	1	1	1
6	2	1	0	1	6	7	7	7	7	7
10	3	2	0	1	6	10	11	16	17	17
16	5	3	0	1	6	10	11	16	17	22

Capacity = 7, Index =3, from the formula discussed above: max(dp[2][7], profit[3] + dp[2][2])

Code

Here is the code for our bottom-up dynamic programming approach:

Java

```java
public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // basic checks
    if (capacity <= 0 || profits.length == 0 || weights.length != profits.length)
      return 0;

    int n = profits.length;
    int[][] dp = new int[n][capacity + 1];

    // populate the capacity=0 columns, with '0' capacity we have '0' profit
    for(int i=0; i < n; i++)
      dp[i][0] = 0;

    // if we have only one weight, we will take it if it is not more than the capacity
    for(int c=0; c <= capacity; c++) {
      if(weights[0] <= c)
        dp[0][c] = profits[0];
    }

    // process all sub-arrays for all the capacities
    for(int i=1; i < n; i++) {
```

Test Case | Run Code Result | Stdout

```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

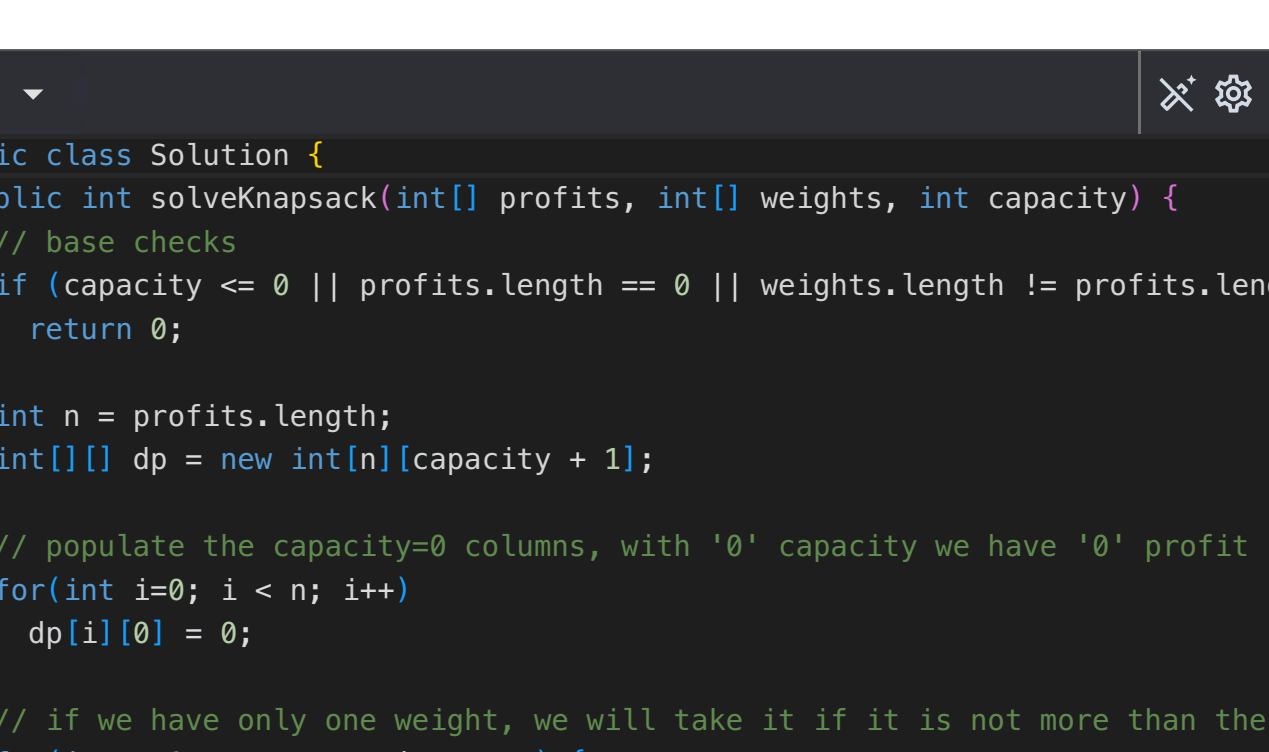
The above solution has a time and space complexity of $O(N * C)$, where 'N' represents total items, and 'C' is the maximum capacity.

How to find the selected items?

As we know that the final profit is at the bottom-right corner; therefore, we will start from there to find the items that will be going in the knapsack.

As you remember, at every step, we had two options: include an item or skip it. If we skip an item, then we take the profit from the remaining items (i.e., from the cell right above it); if we include the item, then we jump to the remaining profit to find more items.

Let's understand this from the above example:


capacity -->

profit	weight	index	0	1	2	3	4	5	6	7
1	1	0 (A)	0	1	1	1	1	1	1	1
6	2	1 (B)	0	1	6	7	7	7	7	7
10	3	2 (C)	0	1	6	10	11	16	17	17
16	5	3 (D)	0	1	6	10	11	16	17	22

1. '22' did not come from the top cell (which is 17); hence we must include the item at index '3' (which is the item 'D').
2. Subtract the profit of item 'D' from '22' to get the remaining profit '6'. We then jump to profit '6' on the same row.
3. '6' came from the top cell, so we jump to row '2'.
4. Again, '6' came from the top cell, so we jump to row '1'.
5. '6' is different than the top cell, so we must include this item (which is item 'B').
6. Subtract the profit of 'B' from '6' to get the profit '0'. We then jump to profit '0' on the same row. As soon as we hit zero remaining profit, we can finish our item search.
7. So items going into the knapsack are {B, D}.

Let's write a function to print the set of items included in the knapsack.

Java

```java
public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // basic checks
    if (capacity <= 0 || profits.length == 0 || weights.length != profits.length)
      return 0;

    int n = profits.length;
    int[][] dp = new int[n][capacity + 1];

    // populate the capacity=0 columns, with '0' capacity we have '0' profit
    for(int i=0; i < n; i++)
      dp[i][0] = 0;

    // if we have only one weight, we will take it if it is not more than the capacity
    for(int c=0; c <= capacity; c++) {
      if(weights[0] <= c)
        dp[0][c] = profits[0];
    }

    // process all sub-arrays for all the capacities
    for(int i=1; i < n; i++) {
```

Test Case | Run Code Result | Stdout

```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

Challenge

Can we further improve our bottom-up DP solution? Can you find an algorithm that has $O(C)$ space complexity?

`We only need one previous row to find the optimal solution!`

Java

```java
public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // TODO: Write your code here
    return -1;
  }
}

```

Test Case | Run Code Result | Stdout

```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

Solution code

Here is the solution code.

Java

```java
public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // basic checks
    if (capacity <= 0 || profits.length == 0 || weights.length != profits.length)
      return 0;

    int n = profits.length;
    // we only need one previous row to find the optimal solution, overall we need '2' r
    // the above solution is similar to the previous solution, the only difference is th
    // we use `i%2` instead if `i` and `(i-1)%2` instead if `i-1`
    int[][] dp = new int[2][capacity+1];

    // if we have only one weight, we will take it if it is not more than the capacity
    for(int c=0; c <= capacity; c++) {
      if(weights[0] <= c)
        dp[0][c] = dp[1][c] = profits[0];
    }

    // process all sub-arrays for all the capacities
    for(int i=1; i < n; i++) {
      for(int c=0; c <= capacity; c++) {
```

Test Case | Run Code Result | Stdout

```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

The above solution is similar to the previous solution; the only difference is that we use `i%2` instead of `i` and `(i-1)%2` instead of `i-1`. This solution has a space complexity of $O(2 * C) = O(C)$, where 'C' is the knapsack's maximum capacity.

This space optimization solution can also be implemented using a single array. It is a bit tricky though, but the intuition is to use the same array for the previous and the next iteration!

If you see closely, we need two values from the previous iteration: `dp[c]` and `dp[c-weight[i]]`

Since our inner loop is iterating over `c:0-->capacity`, let's see how this might affect our two required values:

1. When we access `dp[c]`, it has not been overridden yet for the current iteration, so it should be fine.
2. `dp[c-weight[i]]` might be overridden if "weight[i] > 0". Therefore we can't use this value for the current iteration.

To solve the second case, we can change our inner loop to process in the reverse direction: `c:capacity-->0`. This will ensure that whenever we change a value in `dp[]`, we will not need it anymore in the current iteration.

Can you try writing this algorithm?

Java

```java
public class Solution {
  public int solveKnapsack(int[] profits, int[] weights, int capacity) {
    // basic checks
    if (capacity <= 0 || profits.length == 0 || weights.length != profits.length)
      return 0;

    int n = profits.length;
    int[] dp = new int[capacity + 1];

    // if we have only one weight, we will take it if it is not more than the
    // capacity
    for (int c = 0; c <= capacity; c++) {
      if (weights[0] <= c)
        dp[c] = profits[0];
    }

    // process all sub-arrays for all the capacities
    for (int i = 1; i < n; i++) {
      for (int c = capacity; c >= 0; c--) {
        int profit1 = 0, profit2 = 0;
        // include the item, if it is not more than the capacity
```

Test Case | Run Code Result | Stdout

```
[1,6,10,16]
[1,2,3,5]
7
```

Use Example Testcases | Run

← Previous
0/1 Knapsack

Next →
Equal Subset Sum Partition

☐ Mark as Completed